

3D Brain Vessel Segmentation from CTA

Author

2026-05-10

0.1 Motivation, Data, and Impact

Cerebrovascular segmentation from CTA supports stroke assessment, aneurysm analysis, surgical planning, and patient-specific hemodynamic modeling. We chose this topic because it combines core computer vision problems—3D segmentation, class imbalance, registration-sensitive preprocessing, and evaluation—with a clinically meaningful application. The task is difficult because vessels occupy only a small fraction of each 3D volume, many structures are 2–4 voxels wide after resampling, anatomy varies across patients, and useful outputs must preserve vessel connectivity rather than only voxel overlap.

We used the TopBrain/TopCoW CTA training data: 25 annotated 3D NIfTI cases with 40 foreground vessel classes plus background. Raw scans were anisotropic and had high HU dynamic range, so preprocessing was run once offline. Images were resampled to 0.6 mm isotropic spacing with linear interpolation, labels with nearest-neighbor interpolation, and intensities were clipped to $[-100, 400]$ HU then scaled to $[0, 1]$. Offline preprocessing avoids repeatedly resampling full volumes inside the dataloader.

0.2 Approach

Full CTA volumes are too large for GPU training, so the model trains on 3D patches. The dataloader uses foreground-aware patch sampling, rare-class targeting, and a minimum center-distance rule so patches contain useful vessel signal without repeatedly sampling the same local region. Rare-class sampling can weight classes by case presence, voxel frequency, or a hybrid of both. Validation uses sliding-window full-volume inference with overlapping patches and probability averaging.

The split file follows the nnU-Net `splits_final.json` format, allowing our baseline and nnU-Net runs to use identical folds. A stratified K-fold option was also added to assign cases by foreground class presence, which helps prevent rare classes from being absent in validation. The pipeline reports class frequency, validation support, and patch hit counts so Dice scores can be interpreted correctly. This matters because absent classes can produce misleading Dice values: if both prediction and ground truth are empty, Dice can appear perfect even though the model learned nothing for that class.

The model is a 3D U-Net with InstanceNorm3d, LeakyReLU, transposed-convolution upsampling, skip connections, and 41 output logits. Additional implemented options include residual blocks, deep supervision, mixed precision, gradient checkpointing, early stopping, and gradient accumulation. The main loss combines Dice and weighted cross-entropy, with optional Tversky and cLDice terms for thin-vessel and topology-sensitive training. Target skeletons can be precomputed so topology losses do not repeatedly skeletonize labels during training.

0.3 Results

The strongest finding was not a new architecture but a data augmentation bug. Early runs showed a two-population pattern in per-class validation curves: midline vessels learned smoothly, while left/right vessel pairs oscillated or stalled. The cause was left/right flipping without remapping label IDs. Because the labelmap distinguishes R-ICA from L-ICA, flipping the left/right axis created contradictory supervision. Removing that flip left anterior/posterior and inferior/superior flips enabled, but stopped corrupting laterality.

This fix explained the main performance jump. Phase 3 used the stronger architecture/loss stack but still had the L/R bug; phase 4 kept the same settings and removed only the unsafe flip. Mean foreground Dice rose from 0.313 to 0.503, and

the all-case foreground Dice rose from 0.404 to 0.660. Bilateral vessel classes improved the most, while already-stable midline controls changed modestly or stayed saturated.

Table 1: Fold-0 validation summary.

Run	Key_change	Mean_FG_Dice	Mean_FG_Dice_All
phase1	CE + Dice baseline	~ 0.27	~ 0.30
phase3_arch	Topology loss + residual/deep supervision	0.313	0.404
phase4_lrfix	Same as phase3, L/R flip removed	0.503	0.660

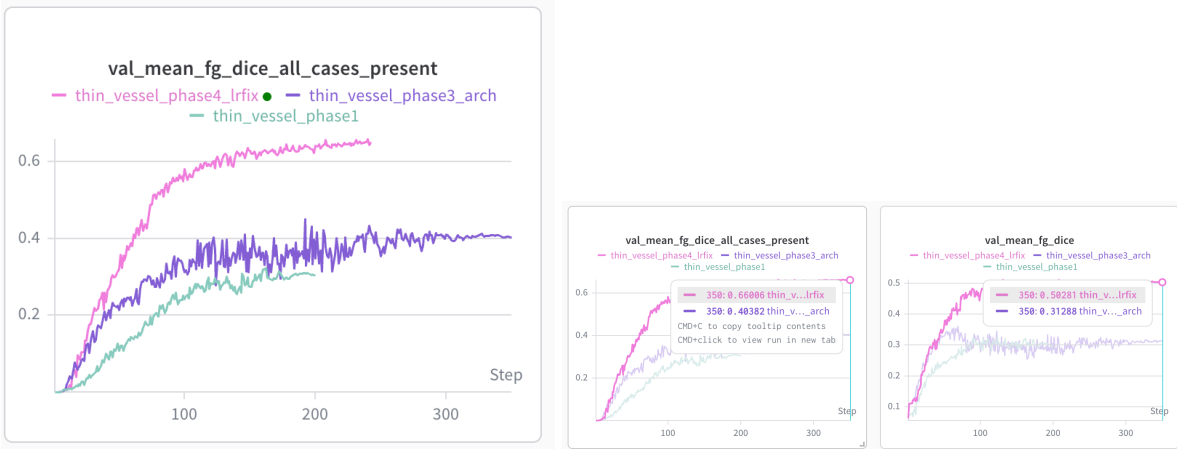


Figure 1: Training curves: phase 4 improves after removing the left/right augmentation bug.

Table 2: Bilateral classes improved most after the L/R flip fix.

Class	Anatomy	Phase3	Phase4
c02	R-P1P2	~ 0.40	~ 0.65
c03	L-P1P2	~ 0.50	~ 0.65
c04	R-ICA	~ 0.37	$\sim 0.65-0.70$
c05	R-M1	~ 0.40	$\sim 0.65-0.70$
c06	L-ICA	~ 0.37	$\sim 0.65-0.70$
c07	L-M1	~ 0.38	$\sim 0.65-0.70$

The per-class W&B panels show why aggregate metrics were insufficient. Phase 3 and phase 4 differ only in the L/R augmentation fix, yet the bilateral vessel panels change from unstable oscillation to smooth convergence. Midline classes act as controls: the Basilar Artery was already near 0.80–0.85, while other midline or venous structures improved modestly after the contradictory L/R signal was removed. Rare posterior-fossa and venous classes remained the hardest, but several moved from near-zero to visibly learning.

0.4 Implementation Details and Resources

The project was implemented in Python with PyTorch, NumPy, SciPy, nibabel, scikit-image, pytest, Weights & Biases, and R Markdown for reporting. The dataset came from the public TopBrain/TopCoW challenge resources, and nnU-Net was used as an external baseline framework. Our original implementation includes the preprocessing workflow, custom dataloader and patch sampler, 3D U-Net training code, validation diagnostics, topology-loss integration, stratified split generation, and tests.

We also integrated nnU-Net as a strong reference baseline. The dataset conversion used the same folds for fair comparison, and a custom trainer path was prepared for topology-aware losses. The nnU-Net fold-0 run reached pseudo-Dice around 0.55–0.60 by epoch 1000, with epoch time near 60 seconds. This gives a useful target for judging whether custom 3D U-Net changes are competitive.

For topology-aware training, soft cIDice was implemented using differentiable morphological skeletonization built from 3D pooling operations. The implementation was checked against a reference cIDice package and a hard skeletonizer used for evaluation. The visualizations below confirm that the soft skeletonization reproduces the reference behavior while the hard skeletonizer produces a thinner centerline.

Table 3: nnU-Net fold-0 summary.

Metric	Value
Approx. pseudo-Dice	0.55–0.60
Epochs	1000
Epoch time	~60 s
Main use	Reference baseline

0.5 Challenge, Innovation, and Contributions

The challenge/innovation score we would expect is **20/20**. This project goes beyond a basic segmentation demo: it implements a full 3D medical image pipeline, adds topology-aware losses, compares against nnU-Net, diagnoses a non-obvious augmentation failure using per-class anatomical evidence, and adds experiment diagnostics that explain why some validation Dice scores are misleading. The major risk was that thin-vessel segmentation can fail even when the code runs correctly, so the project required debugging metrics and data behavior, not just implementing a known model. Each group member contributed to implementation rather than only planning/reporting. Contributions included preprocessing and data inspection, dataloader and sampling code, model/loss/training code, nnU-Net integration, experiment execution, diagnostics, testing, and report writing. The submitted code and README document how the repository is organized and how to run preprocessing, split generation, training, and evaluation.

The project includes pytest coverage for model output shapes, preprocessing, dataloader patch sampling, split validation, loss edge cases, patch-hit diagnostics, split class statistics, thin-vessel metric helpers, early stopping, gradient accumulation, and stratified fold generation. These tests guard the main failure-prone pieces: tensor shapes, absent-class metric behavior, data leakage, and training-control logic.

Additional extensions were implemented but not fully trained because of GPU memory limits: stronger 3D augmentation, SkelRecall, cbDice, CAS connectivity loss, and optional topology/connectivity bottleneck modules. They are disabled by default and remain future work once enough VRAM or further memory optimization is available.

0.6 Conclusion

This project produced a full CTA vessel segmentation pipeline: preprocessing, patch sampling, 3D U-Net training, topology-aware losses, nnU-Net comparison, diagnostics, and automated tests. The central lesson is that multi-class medical segmentation must be debugged at the per-class anatomical level. Aggregate Dice only showed that training was poor; per-class grouping revealed that laterality-specific labels were being corrupted by unsafe augmentation. Fixing that issue gave the largest single improvement, raising mean foreground Dice from roughly 0.31 to 0.50 without changing the architecture or loss.

The most important remaining work is to improve rare and thin vessels through stratified validation, targeted sampling, topology-aware objectives, memory-efficient training, and full-volume challenge-style evaluation across folds.

0.7 Figures and References

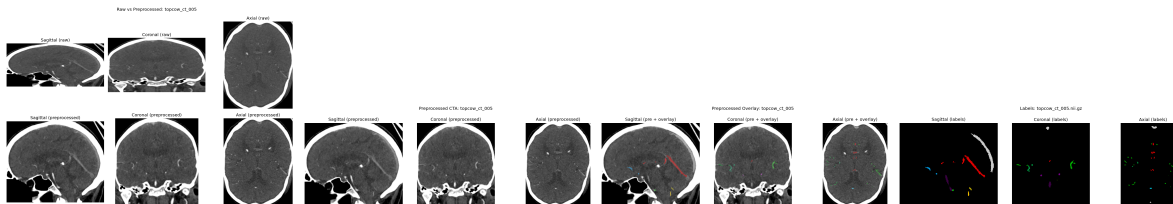


Figure 2: Data checks after preprocessing: raw/resampled CTA, resampled slices, label overlay, and class imbalance.

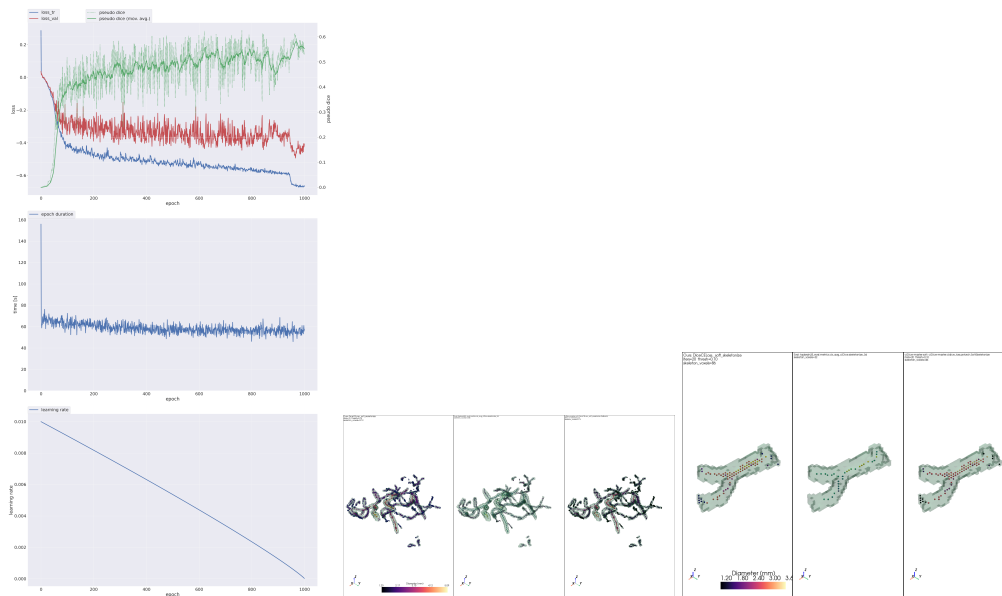


Figure 3: nnU-Net training progress and centerline extraction checks.



Figure 4: Per-class validation Dice panels from W&B.

0.8 References

- [1] Yang K., Musio F., Ma Y., Juchler N., Paetzold J.C., et al. *Benchmarking the CoW with the TopCoW Challenge*. arXiv:2312.17670, 2023.
- [2] Ronneberger O., Fischer P., Brox T. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI, 2015.
- [3] Cicek O., Abdulkadir A., Lienkamp S.S., Brox T., Ronneberger O. *3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation*. MICCAI, 2016.
- [4] He K., Zhang X., Ren S., Sun J. *Deep Residual Learning for Image Recognition*. CVPR, 2016.
- [5] Shit S., Paetzold J.C., Sekuboyina A., Ezhov I., Unger A., Zhylka A., Pluim J.P.W., Bauer U., Menze B.H. *clDice: A Novel Topology-Preserving Loss Function for Tubular Structure Segmentation*. CVPR, 2021.
- [6] Salehi S.S.M., Erdogmus D., Gholipour A. *Tversky Loss Function for Image Segmentation Using 3D Fully Convolutional Deep Networks*. MLMI, 2017.
- [7] Isensee F., Jaeger P.F., Kohl S.A.A., Petersen J., Maier-Hein K.H. *nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation*. Nature Methods, 2021.